NEXT ITERATION DESIGN

Task 2: Process Flow and Sprint Implementation Plan

Internal Task Portal, Nexus AML

Scope: Assignment, filtering, task details, evidence submission, admin review, and completion approval.

Prepared as the Part 2 design deliverable

Table of Contents

1	Scope and working assumptions	3
2	Process flow map	4
3	Proposed solution	5
4	Sprint implementation plan	6
5	Dependency sequence and critical path	10
6	Risks and mitigations	10
7	Definition of done	10

1. Scope and Working Assumptions

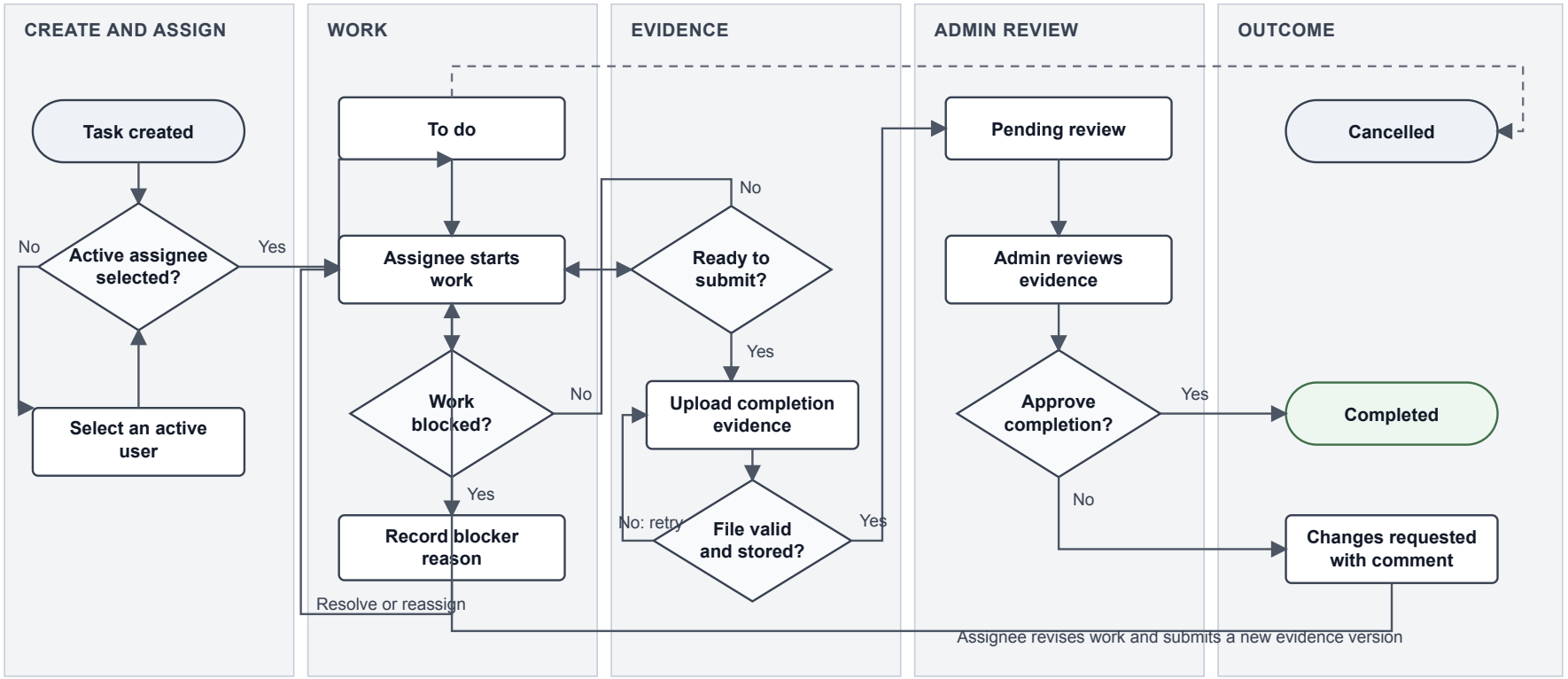
- The portal begins as one internal workspace. Active users can discover shared tasks, while write permissions depend on role and assignment.
- Two roles are sufficient for this iteration: **User** and **Admin**. Admins retain normal user capabilities.
- A task has one assignee at a time. Reassignment is allowed before completion and is recorded in the activity history.
- Only the current assignee can submit or replace evidence. Only an admin can approve or reject it.
- Evidence is stored in a private object store. The relational database stores metadata and object keys, never public URLs or file contents.
- Accepted evidence formats are PDF, PNG, and JPEG, with a configurable 10 MB limit. This must be confirmed with the product owner.
- Rejection requires an admin comment and returns the task to the assignee. Previous evidence and decisions remain in the audit history.
- A task becomes **Completed** only through an approved review. Direct status changes to Completed are removed.
- Product decision required: whether an admin may approve their own evidence. The safer default is to prohibit self-approval.

Proposed Task States

Status	Meaning
To do	Created and assigned, but work has not started.
In progress	The assignee is actively working.
Blocked	Progress cannot continue; a reason is required.
Pending review	Evidence has been submitted and is locked for review.
Changes requested	An admin rejected the submission with feedback.
Completed	An admin approved the evidence.
Cancelled	Work is no longer required; evidence cannot be submitted.

2. Process Flow Map

Solid lines show the normal workflow and recovery loops. Dashed lines show administrative cancellation.



Assignment: A task enters the active queue only after an active user is selected.

Blocked work: A reason is required. Resolution or reassignment returns the task to active work.

Evidence: Invalid files and failed scans leave the task in its current state and provide a retry path.

Submission: Verified evidence is recorded before the task moves to Pending review.

Rejection: A comment is required. The assignee can revise the work and submit a new evidence version.

Completion: Approval and completion occur in one transaction. Cancellation requires admin authorization.

3. Proposed Solution

Data Model Changes

- Add **role** to User, initially USER or ADMIN.
- Add **assigneeId**, **createdById**, **status**, and **version** to Task. The version supports optimistic concurrency.
- Add Evidence with task, uploader, private object key, original filename, MIME type, size, checksum, version, and timestamp.
- Add Review with evidence, reviewer, decision, comment, and decision timestamp.
- Add TaskActivity as an append-only record of assignment and workflow transitions.
- Add indexes for status, assignee, and combined status plus assignee filters.

API and Authorization Boundaries

Method	Endpoint	Purpose and authorization
GET	`/api/tasks?status=&assigneeId=`	Return visible tasks using combinable, server-side filters.
GET	`/api/tasks/:taskId`	Return task details, evidence metadata, review history, and permitted activity.
POST	`/api/tasks/:taskId/evidence/upload-url`	Authorize the current assignee and return a short-lived upload instruction.
POST	`/api/tasks/:taskId/evidence`	Verify the uploaded object and submit its metadata for review.
POST	`/api/tasks/:taskId/reviews`	Allow an admin to approve or reject using the expected task version.

File Security and Operational Controls

- Use a private bucket with encryption at rest, blocked public access, and short-lived signed URLs.
- Validate declared MIME type, detected signature, extension, size, and checksum. Quarantine uploads until malware scanning succeeds.
- Generate object keys server-side and bind them to user and task context.
- Record upload, submission, review, reassignment, and completion in an append-only audit trail.
- Confirm retention and deletion requirements with compliance before release.

4. Sprint Implementation Plan

Delivery assumptions: Two-week sprints, two full-stack engineers, and shared QA and product support. Story points indicate relative complexity.

Sprint 1: Shared Work, Assignment, and Task Details

> **Goal:** Users can discover relevant shared work, filter it, assign it, and open a secure task details screen.

T2-01: Define Roles and Authorization Policy (3 points)

Dependencies: None.

Implementation:

- Add User and Admin roles with one centralized permission matrix.
- Document a safe first-admin mechanism.
- Add negative authorization tests for every role and unauthenticated callers.

Acceptance criteria:

- Unauthenticated requests return 401; forbidden requests return 403.
- Changing payloads or route parameters cannot grant admin capability.

T2-02: Extend Task and Assignment Schema (5 points)

Dependencies: T2-01 policy decisions.

Implementation:

- Add assignee, creator, expanded status, and optimistic version fields through Prisma migration.
- Backfill existing tasks to their creator as assignee.
- Add indexes for status, assignee, and combined filters.

Acceptance criteria:

- The migration succeeds on production-like data and has a forward-fix plan.
- Existing tasks remain accessible and invalid state changes are rejected.

T2-03: Add Assignment Workflow (3 points)

Dependencies: T2-01 and T2-02.

Implementation:

- Add active-user lookup and assignment controls to task create and edit flows.
- Record assignment and reassignment in TaskActivity.

Acceptance criteria:

- Authorized users can assign one active user.
- Disabled or unknown users cannot be assigned.

T2-04: Implement Server-side Task Filtering (5 points)

Dependencies: T2-02.

Implementation:

- Add status and assignee query parameters to the API and repository.
- Represent filter state in the URL and define loading, empty, and clear-filter states.

Acceptance criteria:

- Filters work independently and together, and survive refresh.
- Filtering occurs in the database; invalid values return 400.

T2-05: Build the Task Details Screen (5 points)

Dependencies: T2-02 and T2-03.

Implementation:

- Add `/tasks/[taskId]` with task, assignee, status, evidence area, and activity placeholders.
- Link list rows to the details route and add secure not-found and forbidden states.

Acceptance criteria:

- Each task has a directly addressable, server-authorized details page.
- Keyboard navigation, headings, focus states, and breadcrumbs are accessible.

Sprint 1 Exit Criteria

- Product accepts assignment, combined filtering, and task-details journeys.
- Authorization tests pass and the migration rehearsal succeeds in staging.

Sprint 2: Evidence Submission and Admin Approval

> **Goal:** An assignee can submit secure evidence and an admin can approve or reject completion with a complete audit history.

T2-06: Provision Private Evidence Storage (5 points)

Dependencies: Approved file policy and infrastructure access.

Implementation:

- Configure private storage, encryption, lifecycle rules, CORS, and service identity.
- Integrate malware scanning and document failed-scan handling.

Acceptance criteria:

- Objects cannot be fetched publicly.
- Upload and download permissions are short-lived and object-specific.

T2-07: Implement Authorized Evidence Upload (5 points)

Dependencies: T2-01, T2-05, and T2-06.

Implementation:

- Add upload authorization and completion endpoints.
- Validate type, signature, size, checksum, task state, and current assignee.
- Show progress, retry, cancel, and validation errors.

Acceptance criteria:

- Only the current assignee can upload for an eligible task.

- Invalid files do not create usable evidence records.

T2-08: Add Submission and Workflow Transitions (5 points)

Dependencies: T2-02 and T2-07.

Implementation:

- Centralize allowed state transitions in a domain service.
- Submit verified evidence and move to Pending review transactionally.
- Retain earlier evidence versions after Changes requested.

Acceptance criteria:

- A task without verified evidence cannot enter Pending review.
- Stale concurrent updates return a conflict and do not overwrite newer work.

T2-09: Build Admin Review and Decision Flow (8 points)

Dependencies: T2-01, T2-05, and T2-08.

Implementation:

- Add an admin-only Pending review queue and evidence viewer.
- Implement approval and rejection; require a comment for rejection.
- Write the review, activity, and task transition in one transaction.

Acceptance criteria:

- Non-admin callers receive 403 from review endpoints.
- Approval is idempotent and is the only normal route to Completed.
- Rejection returns the task to Changes requested with visible feedback.

T2-10: Complete Audit and Observability (3 points)

Dependencies: T2-03, T2-08, and T2-09.

Implementation:

- Show a chronological activity timeline.
- Add in-app notifications for assignment, rejection, and approval.
- Add structured logs and metrics without logging sensitive file data.

Acceptance criteria:

- Every transition records actor, timestamp, prior state, and new state.
- Support staff can correlate a failed UI action with a server request.

T2-11: Security, Accessibility, and Release Validation (5 points)

Dependencies: T2-06 through T2-10.

Implementation:

- Test happy paths, rejection and resubmission, horizontal access, malicious files, expired links, and concurrent revie
- Review keyboard navigation, upload progress, evidence links, and decision feedback.
- Rehearse migration, rollout, and rollback in staging.

Acceptance criteria:

- Critical workflows pass in supported browsers and keyboard-only navigation.

- No object is public and no signed URL is persisted or logged.
- Product, engineering, QA, and security approve the release checklist.

Sprint 2 Exit Criteria

- The create-to-approval flow is accepted in staging.
- Audit records reconcile with tested transitions.
- Monitoring, support notes, feature flags, and rollback steps are ready.

5. Dependency Sequence and Critical Path

1. Confirm roles, assignment rules, file policy, retention, and self-approval policy.
2. Apply the task and authorization migration.
3. Deliver assignment, filtering, and the details route.
4. Provision private storage and malware scanning.
5. Implement upload authorization before submission transitions.
6. Implement submission transitions before admin review actions.
7. Add audit history and end-to-end tests before release.

Critical path: Authorization policy → schema migration → task details → storage → evidence submission → admin decision → release validation.

6. Risks and Mitigations

Risk	Mitigation
Sensitive evidence exposure	Private storage, short-lived signed URLs, least-privilege credentials, and negative security tests.
Unsafe file content	Signature validation, quarantine, malware scanning, and no inline rendering of untrusted formats.
Conflicting updates	Task version checks and transactional review decisions.
Authorization drift	One permission matrix and negative integration tests.
Migration regression	Backfill rehearsal with production-like data and staged rollout.
Unbounded storage	File limits, lifecycle rules, orphan cleanup, retention policy, and monitoring.

7. Definition of Done

- Acceptance criteria and automated tests pass in CI.
- Prisma migrations are reviewed, repeatable, and verified in staging.
- Authorization is enforced and negatively tested at every API boundary.
- Evidence storage is private, scanned, encrypted, and covered by retention rules.
- Accessibility checks pass for keyboard navigation, focus, status feedback, and upload progress.
- Logs, metrics, feature flags, rollback steps, and support notes are ready.
- README and API documentation reflect the released workflow.

Part 2 is a design and planning deliverable. These changes are not implemented in the Part 1 application.